University Master's Degree in
Machine Learning ML

Assignment: Fundamentals of Continuous Integration

The main objective of this assignment is to start understanding the fundamentals of Continuous Integration (CI). Specifically, we will analyze some of the basic elements which are the linting, search for syntax errors to guarantee code correctness, automatic code formatting (black) and testing, to ensure the code functionality.

You will include the lab code in a public GitHub repository (named Lab0), where you will store all the developed functionalities.

To follow the best practices of MLOps, we will start by using a **specific virtual environment for the project**. Therefore, you will have to create a new environment (*uv init* and *uv sync*) and install the specific dependencies (*uv add library-name*) for this project, which are:

- Click: package to develop the command line interface for the logic of the project.
- Black: package to format the code.
- Pylint: package to perform the linting of the python files associated with the project.
- Pytest: framework to perform the testing of the project (both unit tests and integration tests)
- Pytest-cov: plugin to measure the coverage of the testing process.

The **structure of the project** must be composed, at least, of:

- A source folder (*src*) where the logic of the project and the implementation of the CLI are stored
- A test folder (*tests*) where the testing files are stored
- The README.md file
- The pyproject.toml configuration file

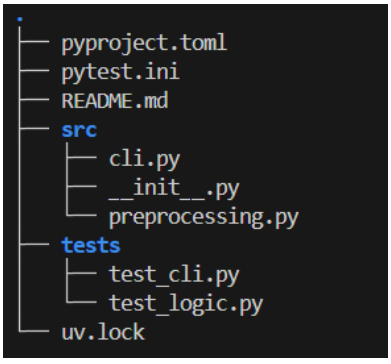

Figure 1. Scaffold of the project.

The **main logic of the project** (named *preprocessing.py* in the previous figure) will be devoted to programing several data pre-processing techniques for different types of data. **The following functionalities must be programmed**:

- Removal of missing values
    - Input: List of values, including missing values like None, '' and nan

    - Output: list of values removing the missing ones
- Filling missing values
    - Inputs
        - List of values, including missing values
        - The value to fill the missing values (default to 0)
    - Output: list of values where the missing values are replaced with the given filling value
- Removal of duplicated values
    - Input: List of values
    - Output: List of unique values
- Normalization of numerical values using the min-max method
    - Inputs:
        - List of numerical values
        - New minimum (default to 0.0)
        - New maximum (default to 1.0)
    - Output: list of normalized values
- Standardization of numerical values using the z-score method
    - Input: List of numerical values
    - Output: List of standardized values
- Clipping numerical values to a certain range
    - Inputs:
        - List of numerical values
        - Minimum value to clip
        - Maximum value to clip
    - Output: List of clipped values
- Conversion of values to integers
    - Input: List of strings (it can include both numerical and non-numerical values)
    - Output: List of values converted to integers (non-numerical values are excluded from the output)
- Logarithmic scale transformation
    - Input: List of values
    - Output: List of values converted to logarithmic scale (just the original positive numbers)
- Tokenization of text into words, selecting only alphanumeric characters and lower-casing words
    - Input: Text to be processed
    - Output: Processed text
- Selection of alphanumerical characters and spaces
    - Input: Text to be processed
    - Output: Processed text
- Removal of stop-words from text (it should be lower-cased)
    - Inputs
        - Text to be processed
        - Stop words to be removed
    - Output: Processed text

- Flatten a list of lists
    - Input: A list of lists
    - Output: A flattened list
- Random shuffling of a list of values
    - Inputs
        - List of values
        - Seed to ensure reproducibility
    - Output: list of shuffled values

Once you have programmed the logic of the functionality of the project you can develop the **command line interface to interact with these functionalities**. To do it, you must use the click package. Specifically, **you must**

- Create **a command for each function** of the main logic programmed before
    - Each command should have a description and an example to show its usage
- **Create a general group named cli**
- **Create four groups associated with the general cli group** (each group should have its description)
    - **Clean**: it will include the **functions related to data cleaning**
        - Removing missing values: it must have an input argument accepting a list of string values
        - Filling missing values: it must have an input string argument accepting a list of string values and an optional argument (option), with 0 as default value and a description of its purpose (help)
    - **Numeric**: it will include **functions related to dealing with numerical attributes**
        - Normalize: it must have an argument to pass the list of numbers and two options to determine the minimum and maximum values of the new range (defaults to 0 and 1, respectively)
        - Standardize: it must have an argument to pass the list of numbers
        - Clip: it must have an argument to pass the list of numbers and two options to determine the minimum and maximum values to clip (defaults to 0 and 1, respectively)
        - Conversion to integers: it must have an argument to pass the list of numbers
        - Transformation to logarithmic scale: it must have an argument to pass the list of numbers
    - **Text**: it will include the **functions to deal with textual information**
        - Tokenization: it must have an input argument accepting string
        - Remove punctuation: it must have an input argument accepting a string
        - Removal of stop-words: it must have an input argument accepting a string and an option to specify the list of strings conforming the stop-words to remove
    - **Struct**: it will include **functions related to the structure of data**
        - Shuffle: it must have an argument to pass the list of values and an option to determine integer value of the seed (default to None)
        - Flatten: it must have an argument to pass the list of lists

        - Unique values: it must have an argument to pass the list of values

Once you have programmed both files, or just one of them, you can use the pylint package to **lint the code**. **Pylint** checks for errors, enforces a coding standard, looks for code smells, and can make suggestions about how the code could be refactored. To run the linting of the project you should execute the following command

uv run python -m pylint src/.py*

Furthermore, you can also **use a formatting tool to standardize the code appearance**. A Python library devoted to this end is black. You can run it executing the following command

uv run black src/.py*

Finally, you must **develop the testing files**. To do it you have to use the pytest framework. Specifically, **you must perform**

- **Unit testing** of all the functionalities developed in the python file where you programmed the logic of the project
    - You must **use at least one fixture** (for instance to generate a list of numbers you can use in several tests)
    - You must **use the parametrize decorator**, which allows one to execute the same test function several times with different values. You should use it at least in one functionality without required options and in another one with required options
- **Integration testing** between the logic and the CLI (you can find information to test CLIs at https://click.palletsprojects.com/en/latest/testing/). That is, testing the outputs of the CLI as it internally uses the logic of the project
    - You must use **a fixture to instantiate the CliRunner**, that will be shared by all the tests by sending it as an input argument. The CliRunner object allows one to invoke the different commands developed in the CLI. Remember that these commands belong to different groups.

You must include both testing files in the tests folder. The name of the testing files must start with *test_* and the name of each test inside the testing files also must start with *test_*. You can run the testing of the project with the following command

uv run python -m pytest -v

You can also use the pytest-cov plugin to analyze the **coverage of the testing process**. To run it you would have to execute the following command

uv run python -m pytest -v --cov=src

Deliverables of the lab

- The link to the GitHub repository
- A report where you
    - Describe the logic you have thought of to test the functionalities of the project
    - Show the results of the linting, formatting and testing processes